Multimeter

OFF
DCV
ACV
DCA
ACA
HFE
N P N
P N P
E B C E
E B C E
VΩ
COM
mA
20A
1000VDC
750VAC
MAX
500V MAX
250 mA
MAX
20A
MAX
MECO
603

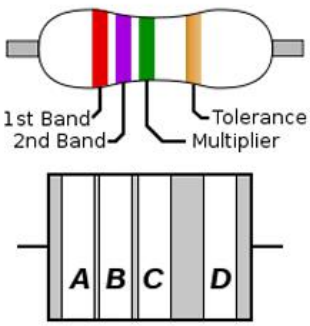
UNITY
DT9205A
AUTO POWER OFF
PNP
NPN
OFF
hFE
20A
mA
COM
V/Ω
MAX 20A
UNFUSED
MAX
1000V

MECO
HOLD
DMM 63
LIGHT
OFF
CAT II
HFE
E B C E
N
P
E B C E
10A
COM
VΩmA
10A MAX
UNFUSED
750VAC
1000VDC
MAX
200mA MAX

1000V
OFF
750V
10A
hFE
NPN
PNP
HAOYUE
DT830D
DIGITAL
MULTIMETER
VΩmA
COM

Resistors

It is a passive component which exhibits a controlled value of resistance across its two leads. Resistance, by definition, is the opposition to the flow of current offered by a conductor, device or circuit. Most axial resistors use a pattern of colored stripes to indicate resistance. A 4-band identification is the most commonly used color-coding scheme on all resistors. It consists of four colored bands that are painted around the body of the resistor. Resistor values are always coded in ohms (Ω). The color codes are given in the following table in Fig. 1.



Band Color	Digit	Multiplier	Tolerance
Black	0	1	---
Brown	1	10	±1%
Red	2	100	±2%
Orange	3	1,000	±3%
Yellow	4	10,000	±4%
Green	5	100,000	---
Blue	6	1,000,000	---
Violet	7	10,000,000	---
Gray	8	100,000,000	---
White	9	---	---
Gold	---	0.1	±5%
Silver	---	0.01	±10%
None	---	---	±20%

For example, a resistor with bands of yellow, violet, red, and gold will have first digit 4 (yellow in table below), second digit 7 (violet), followed by 2 (red) zeros: 4,700 ohms. Gold signifies that the tolerance is ±5%, so the real resistance could lie anywhere between 4,465 and 4,935 ohms.

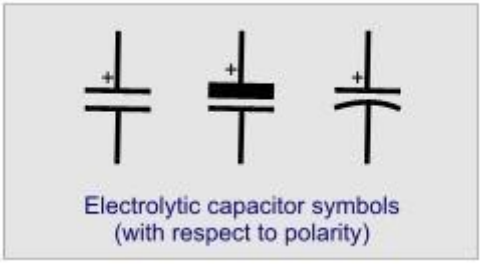
Observation

Obs. No.	Resistance (using Color Code)	Resistance (using multimeter)

Capacitors

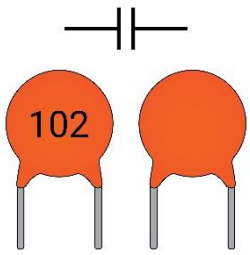
Electrolytic capacitors:

An electrolytic capacitor is a type of capacitor that uses an electrolyte, an ionic conducting liquid, as one of its plates, to achieve a larger capacitance per unit volume than other types. They are used in relatively high current and low-frequency electrical circuits. However, the voltage applied to these capacitors must be polarized; one specified terminal must always have positive potential with respect to the other. These are of two types, axial and radial capacitors as shown in figure below. **The arrowed stripe indicates the polarity, with the arrows pointing towards the negative pin.**



Ceramic capacitors

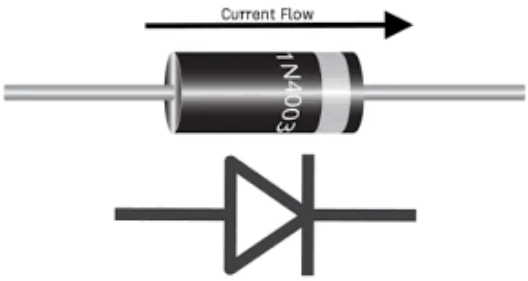
Ceramic capacitors are generally non-polarized and almost as common as radial electrolytic capacitors. Generally, they use an alphanumeric marking system. They may also be written out directly, for instance, 2n2 = 2.2 nF.



Obs. No.	Capacitance (using Code/premarked value)	Capacitance (using multimeter)

Diodes

The term semiconductor diode refers to two electrodes/ terminal device. A semiconductor diode is a one-way device, offering a low resistance when forward biased, and behaving almost as an open switch when reverse biased. A normal pn-junction diode consists of a p-type & n-type semiconductors sandwiched together. The p-side of the diode is always positive & is termed as anode while the n-side of the diode is always negative and termed as cathode. The circuit diagram of the diode is an arrowhead and bar where the arrowhead indicates the conventional direction of current flow under forward biased.

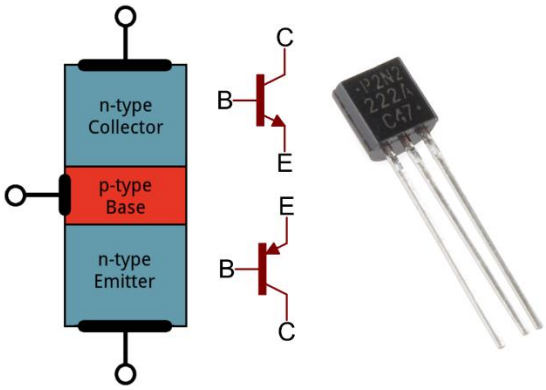


Diodes	Voltage (using Diode mode)
Rectifier	
Zener	

Transistors

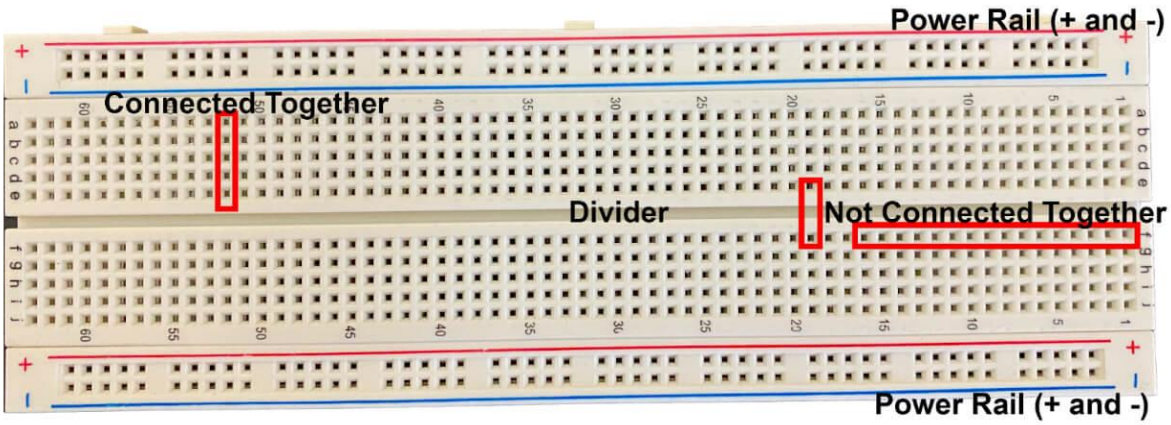
A bipolar junction transistor (BJT) is simply a sandwich of one type of semiconductor material (n/p type) between two layers of the opposite type. Hence, there are basically two configurations of BJT namely; NPN & PNP transistors. A small current at the central region terminal controls the much larger total current flow through the device.

Hence, a transistor can be used for current amplification & voltage amplification. From the above description, it is clear that a BJT is a three terminal device that is, the centre layer is known as base (B) and one of the outer layers is referred as emitter (E) and the other layer as collector (C). The symbolic representations are shown below;



Transistors	NPN / PNP	Voltage

Breadboard



1. Installation of Arduino IDE & Connecting Arduino to Computer

Aim: To install the Arduino IDE on Ubuntu/Windows operating system (OS) and connect the Arduino board to the computer.

Requirements: Arduino (Uno) board, Computer with Windows/Ubuntu OS with access to internet.

Arduino IDE Installation (Ubuntu)

1. Download the AppImage
   Go to https://www.arduino.cc/en/software/ and download the latest .AppImage file.

2. Make a new directory
   ```
   cd~
   mkdir Arduino
   ```

3. Copy the downloaded file to new directory
   ```
   cp ~/Downloads/arduino-ide_2.3.6_Linux_64bit.AppImage ~/Arduino
   ```

   Make sure the files are copied to the location
   ```
   cd Arduino
   ls-la
   ```

4. Make the File executable
   ```
   Chmod +x arduino-ide_2.3.6_Linux_64bit.AppImage
   ```

5. Package Management: Open terminal and type
   ```
   sudo add-apt-repository universe
   sudo apt install libfuse2
   ```

6. To execute the file, type
   ```
   ./arduino-ide_2.3.6_Linux_64bit.AppImage --no-sandbox
   ```
   Arduino IDE will now be open.

7. Type in the following code
   ```
   void setup() {
     Serial.begin(9600);

   }

   void loop() {
   ```

```
    Serial.println(100);
    delay(100);

  }
```

Connect the Arduino Uno to the computer via USB, select the port from the tools menu, verify and upload the code.

You will get an **error** message.

To fix error, you need to set the read/write permission to the corresponding port.

Close Arduino IDE and open terminal and type

```
ls -la /dev | grep ttyACM0
```

8. To confirm the port to set read/write permissions type

```
sudo usermod -a -G dialout <username>
sudo chmod a+rw /dev/ttyACM0
```

9. To confirm change, execute the file, type

```
./arduino-ide_2.3.6_Linux_64bit.AppImage --no-sandbox
```

10. Now try the step 7 again

It worked!! No errors.

In tools menu, select 'Serial Monitor'. You should see the number 100 printed repeatedly, one per line, about 10 times every second.

Creating Application Shortcut

1. Install Editor: open terminal and type

```
sudo apt install gedit
cd ~/.local/share/applications/
ls -la
gedit Arduino.desktop
```

2. Inside the g-editor, type the following:

[Desktop Entry]
Type=Application
Name=Arduino IDE 2.3.6
Exec=/home/user/Arduino/arduino-ide_2.3.6_Linux_64bit.AppImage -–no-sandbox

3. To make a shortcut on desktop, open the folder containing the application, in the terminal type

```
xdg-open /home/user/.local/share/applications/
```

4. Copy the application to desktop using the mouse.

5. Open properties and select allow launching.

6. Now, the Arduino IDE can be run by double-clicking this application icon on the desktop.

Arduino IDE Installation (Windows)

1. **Download**
   Go to arduino.cc/software and download the Windows Installer .exe.

2. **Run Installer**
   Double-click the downloaded file and follow the prompts. Accept defaults and install.

3. **Launch IDE**
   Check Run Arduino IDE after install and click Finish.

4. **Connect Board**
   Plug in your Arduino via USB. Windows installs drivers automatically.

5. **Verify**
   In Arduino IDE, select your board and port under Tools.

   Type in the following code

```
void setup() {
  Serial.begin(9600);

}

void loop() {
  Serial.println(100);
  delay(100);

}
```

   In tools menu, select 'Serial Monitor'. You should see the number 100 printed repeatedly, one per line, about 10 times every second.

---

2. Program to Blink in-built LED on Arduino

Aim: To write a code to blink the in-built LED on Arduino.

Requirements: Arduino (Uno) board, Computer with Arduino IDE installed.

Theory: The Arduino Uno includes a built-in LED connected to digital pin 13. By configuring this pin as an output and toggling its state between HIGH and LOW at timed intervals, the LED can be made to blink. The fundamental functions used are:

- pinMode *(pin, MODE)* to set a pin as INPUT or OUTPUT
- digitalWrite *(pin, VALUE)* to drive the pin HIGH (5 V) or LOW (0 V)
- delay*(ms)* to pause execution for the specified number of milliseconds

Arduino Sketch

```
void setup() {
  pinMode(13, OUTPUT);   // Set pin 13 as output
}

void loop() {
  digitalWrite(13, HIGH);  // Turn LED on
  delay(1000);             // Wait 1 second
  digitalWrite(13, LOW);   // Turn LED off
  delay(1000);             // Wait 1 second
}

```

Procedure

1. Connect the Arduino board to your computer via USB.
2. Open the Arduino IDE and select the correct board (Tools → Board → Arduino Uno) and serial port (Tools → Port → COMx / /dev/ttyACM0).
3. Enter the sketch above into a new IDE window.
4. Verify the code by clicking the “ ”button.
5. Upload the sketch to the board by clicking the “ ” button.

6. Observe the onboard LED (labelled "L") blinking on and off at 1 Hz (500 ms ON, 500 ms OFF).
7. Modify the delay values (e.g., delay(200) / delay(800)) to see the effect on blink rate, then re-upload and observe.

Observation Table

Trial	ON Delay (ms)	OFF Delay (ms)	Observed Blink Rate (Hz)
1	500	500	1.0
2	200	200	2.5
3	1000	1000	0.5

Result

The onboard LED successfully blinked at the programmed intervals, demonstrating control of a digital output pin via Arduino code.

Precautions

- Ensure the correct COM port and board are selected in the IDE.
- Do not short any pins or expose the board to electrostatic discharge.
- Always disconnect power before rewiring external circuits.

Quiz Questions

1. What function sets a pin as an output and why is it necessary?
2. How would you modify the sketch to blink the LED three times quickly, then pause for 2 seconds before repeating?
3. Explain how delay() affects program execution and mention an alternative approach for non-blocking timing.

---

3. Arduino with Tri colour LED and Push Button

Aim: To interface a cathode/anode RGB LED and a pushbutton with an Arduino so that, while the button is held down, the LED blinks Blue → Green → Red in sequence.

Components: Arduino Uno (or compatible), Breadboard, LED (single colour), micro Pushbutton switch, 150 Ω resistors, 10 kΩ resistor, Jumper wires, computer with Arduino IDE and Tri colour LED.

Theory: A common-cathode tricolour LED has a shared cathode (ground) and three separate anodes for red, green, and blue. A common-anode tricolour LED has a shared anode (high) and three separate cathodes for red, green, and blue. The micro pin push button is in ON while pushed operation and bounces back to OFF on release.

Circuit Diagrams

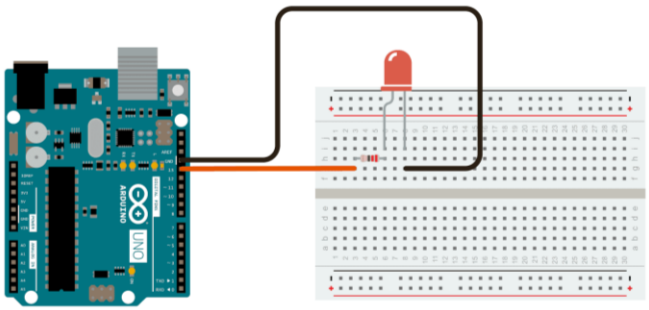


Figure 1.: Connection of LED

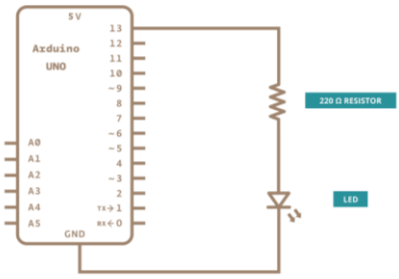


Figure 2.: Circuit Diagram of fig. 1

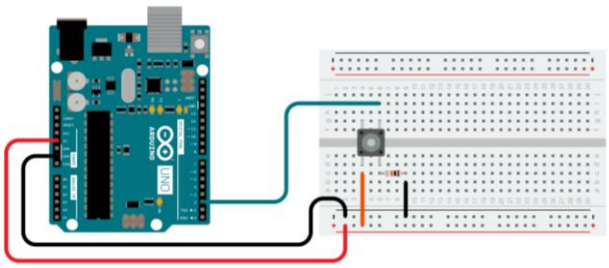


Figure 3: Connection of PushButton

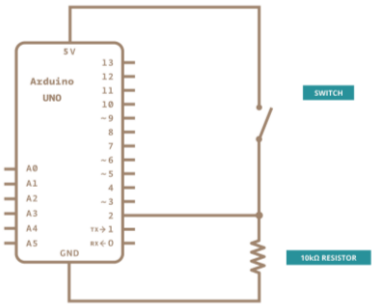


Figure 4: Circuit diagram of fig. 3

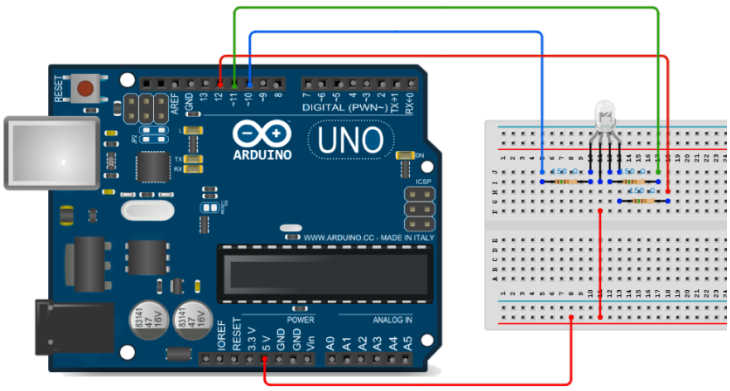


Figure 5: Connection of anode tricolour LED

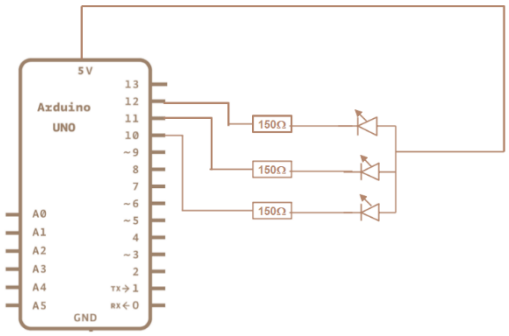


Figure 6: Circuit diagram of fig. 5

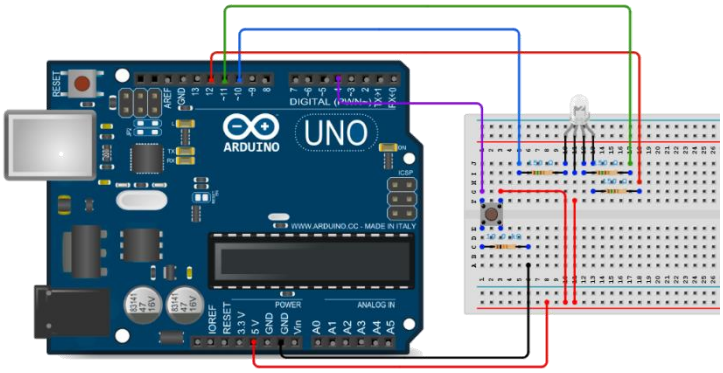


Figure 7: Connection of Pushbutton and anode Tri colour LED

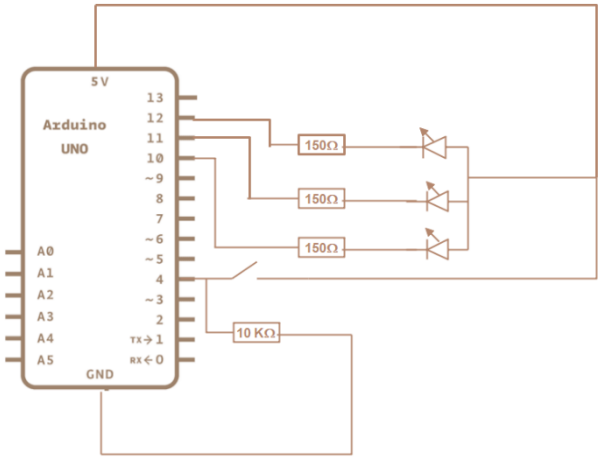


Figure 8: circuit diagram of figure 7.

Arduino Sketch 1

```
void setup() {
  pinMode(10, OUTPUT);   //Blue LED
  pinMode(11, OUTPUT);  // Red LED
  pinMode(12, OUTPUT);  // Green LED

}

void loop() {
    digitalWrite(10, LOW);   //Blue ON
    digitalWrite(11, HIGH);  //Red OFF
    digitalWrite(12, HIGH);  //Green OFF
    delay(1000);
    digitalWrite(11, LOW);  //Red ON
    digitalWrite(10, HIGH); //Blue OFF
```

Arduino Sketch 2

```
void setup() {
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
  pinMode(12, OUTPUT);
  pinMode(4, INPUT);
}


void loop() {
  digitalWrite(10, HIGH);
  digitalWrite(11, HIGH);
  digitalWrite(12, HIGH);
  if (digitalRead(4) == HIGH)
  {
```

```
    digitalWrite(12, HIGH); //Green OFF
    delay(1000);
    digitalWrite(12, LOW);  //Green OFF
    digitalWrite(10, HIGH); //Blue OFF
    digitalWrite(11, HIGH); //Red OFF
    delay(1000);
}
```

```
    digitalWrite(10, LOW);
      digitalWrite(11, HIGH);
      digitalWrite(12, HIGH);
      delay(1000);
      digitalWrite(11, LOW);
      digitalWrite(10, HIGH);
      digitalWrite(12, HIGH);
      delay(1000);
      digitalWrite(12, LOW);
      digitalWrite(10, HIGH);
      digitalWrite(11, HIGH);
      delay(1000);
    }
  }
```

Procedure:

1) Assemble the circuit as shown in figure 6.
2) Connect the Arduino via USB
3) Open the Arduino IDE, set board and port.
4) Type the Arduino sketch 1 in the IDE.
5) Verify and upload the code.
6) Observe the behaviour of the LED
7) Assemble the circuit as shown in figure 9.
8) Type the Arduino sketch 2 in the IDE.
9) Verify and upload the code.
10) Press the pushbutton; the LED should follow the cycle:

    Blue → Green → Red

11) Release the button mid-cycle and note that the remainder of that colour's blink still completes before stopping.
12) Test repeatedly to confirm consistent behaviour.

Observation Table

Button State	Expected LED Behaviour	Actual Behaviour
Not pressed	All LEDs OFF	
Press & hold	Blue blink → Green blink → Red blink repeat	
Release mid-cycle	Completes current colour blink, then OFF	

Result: When the pushbutton is held down, the RGB LED cycles through Blue, Green, and Red blinks (each for1000 ms). Releasing the button stops further cycles after the current blink cycle completes.

Precautions

- Ensure the button input has a solid pull-down to avoid floating reads.
- Use current-limiting resistors on each LED anode to prevent damage.
- Verify pin assignments in code match your wiring.

Quiz Questions

1. Explain why releasing the button mid-sequence does not immediately halt the LED blink.
2. How would you modify the sketch so that the sequence stops instantly when the button is released?

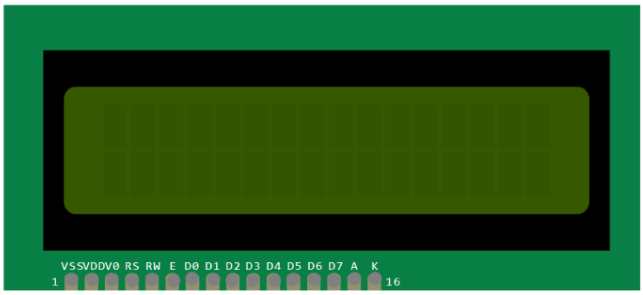
---

4. LCD Interfacing with Arduino

Aim: Connect a 16×2 character LCD to an Arduino (Uno) using 4-bit mode and write a program to display text on Line 1 and Line 2.

Components: Arduino Uno (or compatible), Breadboard and Jumper wires, 10 kΩ potentiometer, computer with Arduino IDE and 16×2 LCD (HD44780).

Theory: A 16×2 character LCD has a controller (HD44780) that accepts commands and data over a parallel interface. In 4-bit mode, only D4–D7 are used for data, reducing pin count. RS selects command/data, E latches the value, and RW is tied to GND for write-only operation.



Pin Map:

LCD Pin	Name	Connect to
1	V_{ss} (Ground)	GND
2	V_{DD} (+5V)	+5V
3	V_0 (Contrast)	Wiper of 10kΩ potentiometer

4	RS (Register Select)	Arduino D12
5	RW (Read/Write)	GND
6	E (Enable)	Arduino D11
7	Data Pin 0	Unused in 4-bit
8	Data Pin 1	Unused in 4-bit
9	Data Pin 2	Unused in 4-bit
10	Data Pin 3	Unused in 4-bit
11	Data Pin 4	Arduino D5
12	Data Pin 5	Arduino D4
13	Data Pin 6	Arduino D3
14	Data Pin 7	Arduino D2
15	LED+	+5V of 10kΩ potentiometer
16	LED-	GND

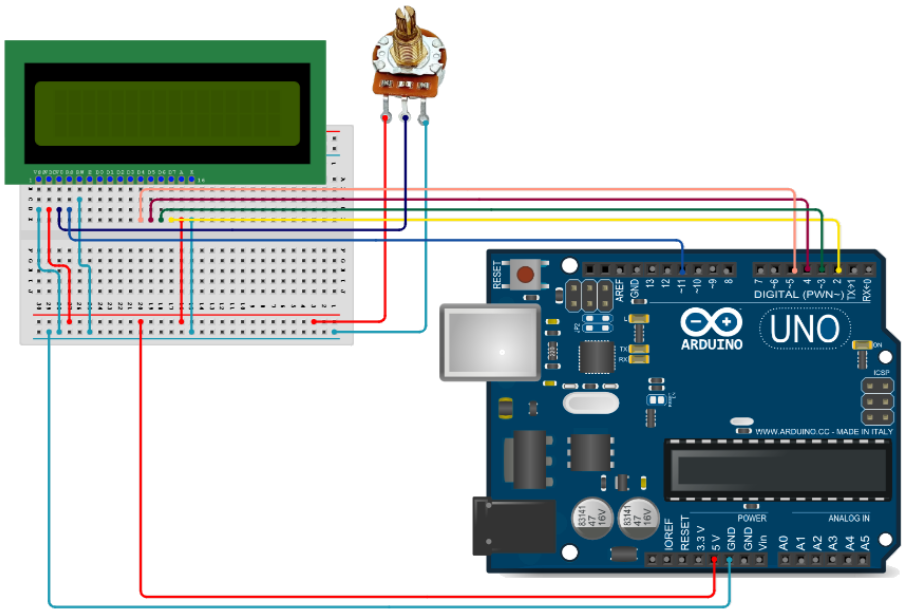


Figure1: Connection of LCD.

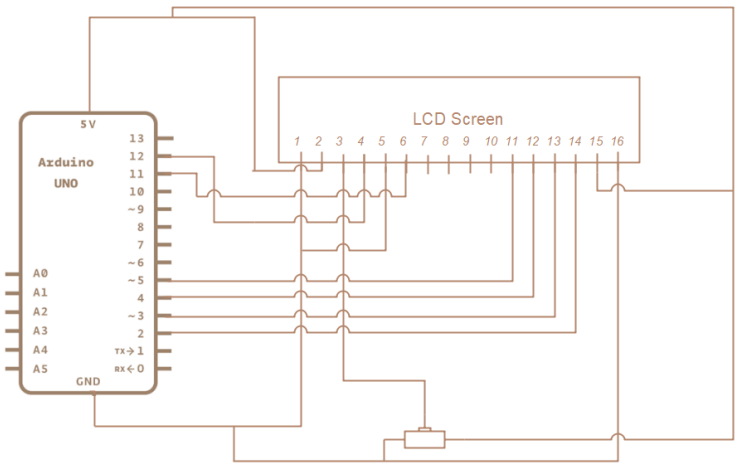


Figure 2: Circuit Diagram of 16 x 2 LCD connection.

Arduino Sketch

```
#include<LiquidCrystal.h>
LiquidCrystal lcd(12,11,5,4,3,2);

void setup(){

lcd.begin(16,2);
lcd.setCursor(0,0);
lcd.print("First row");
lcd.setCursor(0,1);
lcd.print("second row");

}

void loop()
{

}
```

Procedure:

13) Assemble the circuit as shown in figure 1.
14) Connect the Arduino via USB
15) Open the Arduino IDE, set board and port.
16) Type the Arduino sketch in the IDE.
17) Verify and upload the code.
18) Adjust the contrast pot until characters are clearly visible.
19) Carry out the below task and note your observations.

Observation

- The LCD displays _____ on the first line and _____ on the second line

Task:

1) Edit the sketch to print your NAME on the 2nd row (line 2).
2) Place the cursor in the 4th Column.
3) Edit the sketch to print your Name in the 1st row and Roll Number in the 2nd Row.

Result

The 16×2 LCD was successfully interfaced with the Arduino and displayed the programmed text on both lines.

Precaution

Do not carry out connections while Arduino is 'ON'.

Practice Task

Write a sketch that displays "CARMEL SUMMER SCHOOL" on a 16×2 LCD and continuously scrolls the text to the left at a readable speed. Refer to the official Arduino documentation for correct use of the function "scrollDisplayLeft() ".

---

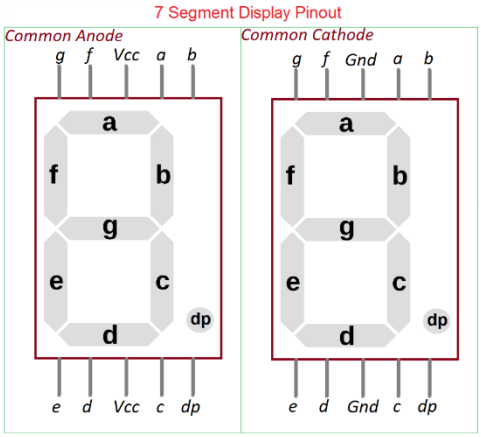
5. Seven Segment Display

Aim: Connect a seven segment display to an Arduino (Uno) and write a program to blink the display and display digits from 0 to 9.

Components: Arduino Uno (or compatible), Breadboard and Jumper wires, seven segment display, 150 Ω resistor and computer with Arduino IDE.

Theory: A seven-segment display consists of 7 LEDs (segments a–g) arranged to form numbers. By selectively turning ON/OFF the LEDs, digits **0–9** can be displayed.

- In **Common Cathode**, all LED cathodes are tied to ground.
- In **Common Anode**, all anodes are tied to +5V, and segments light up with LOW signals.



Pin Map:

Display Pin	Connect to
a	Arduino D2
b	Arduino D3
c	Arduino D4
d	Arduino D5
e	Arduino D6
f	Arduino D7
g	Arduino D8
Dot/dp	Optional (connect to digital pin)
COM	GND (for CC) /+5V (for CA)

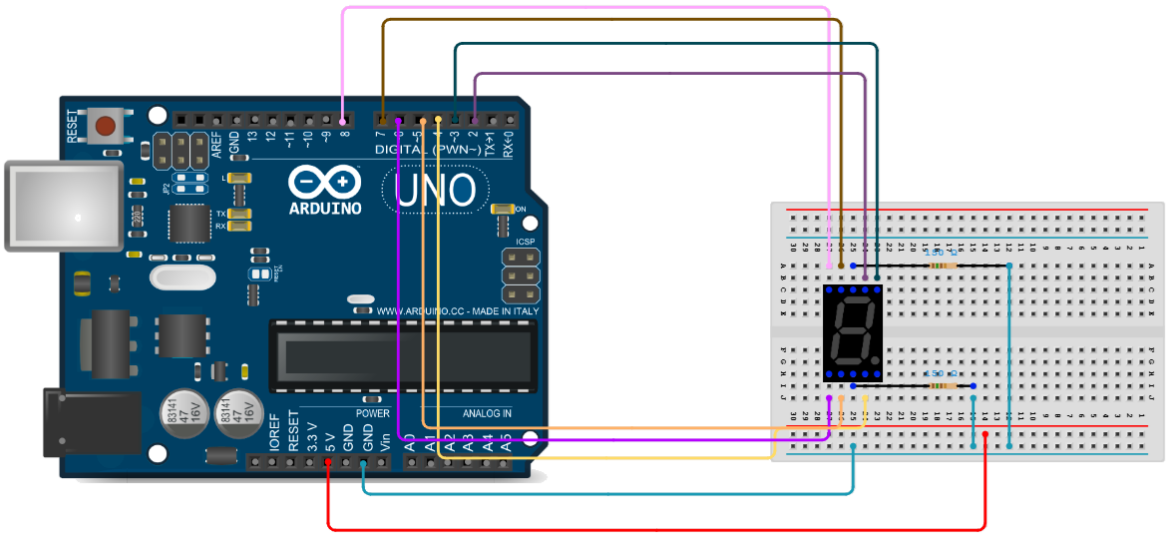


Figure1: Connection of seven segment display(CC).

Arduino Sketch 1

```
// To blink the LEDs on SSD

#define  a 2// a to PIN2
#define  b 3// b to PIN3
#define  c 4// c to PIN4
#define  d 5// d to PIN5
#define  e 6// e to PIN6
#define  f 8// f to PIN8
#define  g 9// g to PIN9

void setup(){
  pinMode(a, OUTPUT);
  pinMode(b, OUTPUT);
  pinMode(c, OUTPUT);
  pinMode(d, OUTPUT);
  pinMode(e, OUTPUT);
  pinMode(f, OUTPUT);
  pinMode(g, OUTPUT);
}

// Code to blink the seven
segment display

void loop(){
  digitalWrite(a,LOW);



digitalWrite(b,LOW);
  digitalWrite(c,LOW);
  digitalWrite(d,LOW);
  digitalWrite(e,LOW);
  digitalWrite(f,LOW);
  digitalWrite(g,LOW);
  delay(1000);
  digitalWrite(a,HIGH);
  digitalWrite(b,HIGH);
  digitalWrite(c,HIGH);
  digitalWrite(d,HIGH);
  digitalWrite(e,HIGH);
  digitalWrite(f,HIGH);
  digitalWrite(g,HIGH);
  delay(1000);
  }
```

Arduino Sketch 2

```
// To display digit from 0
to 9

void loop()
{
//ZERO
digitalWrite(a,HIGH);
digitalWrite(b,HIGH);
digitalWrite(c,HIGH);
digitalWrite(d,HIGH);
digitalWrite(e,HIGH);
digitalWrite(f,HIGH);
digitalWrite(g,LOW);
delay(1000);
//ONE
digitalWrite(a,LOW);
digitalWrite(b,HIGH);
digitalWrite(c,HIGH);
digitalWrite(d,LOW);
digitalWrite(e,LOW);
digitalWrite(f,LOW);
digitalWrite(g,LOW);
delay(1000);
//TWO
digitalWrite(a,HIGH);
digitalWrite(b,HIGH);
digitalWrite(c,LOW);
digitalWrite(d,HIGH);
digitalWrite(e,HIGH);
digitalWrite(f,LOW);
digitalWrite(g,HIGH);
delay(1000);
//THREE
digitalWrite(a,HIGH);
digitalWrite(b,HIGH);
digitalWrite(c,HIGH);
digitalWrite(d,HIGH);
digitalWrite(e,LOW);
digitalWrite(f,LOW);
digitalWrite(g,HIGH);
delay(1000);
//FOUR
digitalWrite(a,LOW);
digitalWrite(b,HIGH);
digitalWrite(c,HIGH);
```

```
digitalWrite(d,LOW);
digitalWrite(e,LOW);
digitalWrite(f,HIGH);
digitalWrite(g,HIGH);
delay(1000);
//Continue the code for
digits Five to Nine
}
```

Procedure:

1) Check if the given display is common anode or common anode
2) Assemble the circuit as shown in figure 1. If your display is common anode, then connect the COM pin to +5V.
3) Connect the Arduino via USB
4) Open the Arduino IDE, set board and port.
5) Type the Arduino Sketch 1 in the IDE.
6) Verify and upload the code.
7) Verify the output of the display.
8) Record your observation.
9) Type Arduino Sketch 2 (For CA-display, modify the Replace LOW with HIGH and vice-versa) and upload
10) Verify the output of the display and record your observations.

Observation

Digit	Segments Glowing (a–g)	Display on 7-segment
0	a, b, c, d, e, f	0
1		
2		
3		
5		
6		
7		
8		
9		

Result

The seven-segment display was successfully interfaced with Arduino, and digits **0–9** were displayed sequentially.

Precaution

Do not carry out connections while Arduino is 'ON'.

Practice Task

Modify the program to:

- Display only even numbers (0,2,4,6,8).
- Display digits in reverse order (9 → 0).
- Display alphabets A, b, C, d, E, F

---

6. Ultrasonic Sensor with Arduino

Aim: To measure the distance of an object using an Ultrasonic sensor (HC-SR04) and display the result on a 16×2 LCD with the help of Arduino.

Components: Arduino Uno (or compatible), Breadboard and Jumper wires, Ultrasonic Sensor, 16 x 2 LCD and computer with Arduino IDE.

Theory: The HC-SR04 Ultrasonic sensor works by sending out an ultrasonic pulse from the trigger pin and receiving the reflected pulse at the echo pin.
The time taken for the echo to return is proportional to the distance of the object.
The distance is then displayed on the 16×2 LCD using the Arduino LiquidCrystal library.

Ultrasonic Ranging Module HC-SR04 provides 2cm-400cm non-contact distance sensing capabilities, Ranging accuracy up to 3mm.

Figure 1: Ultrasonic Sensor

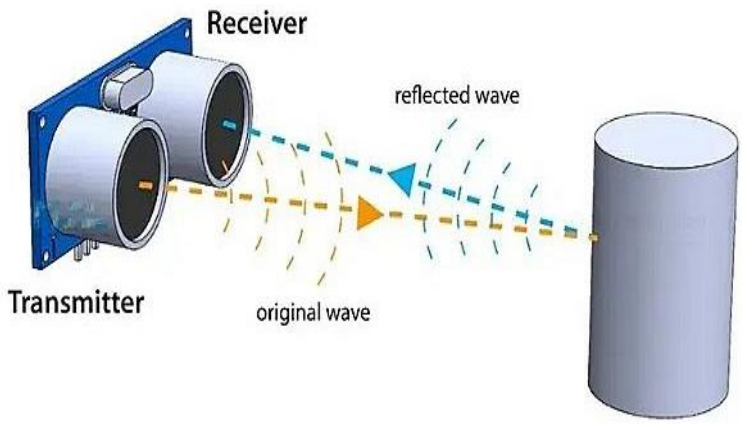


Figure 2: Working Principle of the ultrasonic sensor [https://images.app.goo.gl/yc97nzBgysKoHJdd9]

Formula:

$$Distance = \frac{Time(s)\ \times 343m/s}{2}$$

The speed of sound in air is 343 m/s. The time measured by Arduino is in micro seconds (μs). To get the distance in cm, we need to convert the speed of sound to cm/μs.

The distance formula becomes,

$$\boldsymbol{Distance = \frac{Time\ (\mu s) \times 0.0343\ cm/\mu s}{2}}$$

Pin Map:

Sensor Pin	Connect to
V_{cc}	5V
Trig	Arduino D7
Echo	Arduino D8
GND	GND

.

LCD Pin	Name	Connect to
1	V_{ss} (Ground)	GND
2	V_{DD} (+5V)	+5V
3	V_0 (Contrast)	Wiper of 10kΩ potentiometer
4	RS (Register Select)	Arduino D12
5	RW (Read/Write)	GND
6	E (Enable)	Arduino D11
7	Data Pin 0	Unused in 4-bit
8	Data Pin 1	Unused in 4-bit
9	Data Pin 2	Unused in 4-bit
10	Data Pin 3	Unused in 4-bit
11	Data Pin 4	Arduino D5
12	Data Pin 5	Arduino D4
13	Data Pin 6	Arduino D3
14	Data Pin 7	Arduino D2
15	LED+	+5V of 10kΩ potentiometer
16	LED-	GND

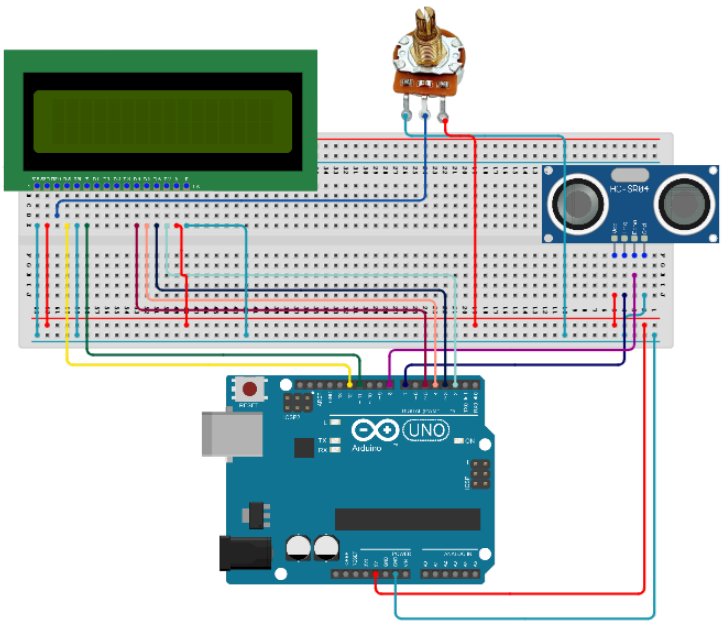


Figure 4: Connection of LCD and Ultrasonic sensor

Arduino Sketch 1

```
#include <LiquidCrystal.h> //Load Liquid Crystal Library
LiquidCrystal LCD(12,11,5,4,3,2); //Create Liquid Crystal Object called lcd

#define trig 7 //Sensor Echo pin connected to Arduino pin 7
#define echo 8 //Sensor Trip pin connected to Arduino pin 8


void setup()
{
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);

  LCD.begin(16,2); //Begin LCD
  LCD.setCursor(0,0);  //Set LCD cursor to upper left corner, column 0, row 0
  LCD.print("Object Distance:");  //Print Message on First Row
}

void loop() {
  float duration, distance;  // Variables
  digitalWrite(trig, LOW);  //digital pin connected to trig is OFF/LOW
  delayMicroseconds(2);        //wait for 2 microseconds
  digitalWrite(trig, HIGH); //digital pin connected to trig is ON/HIGH
  delayMicroseconds(10);       //wait for 10 microseconds
  digitalWrite(trig, LOW);  //digital pin connected to trig is OFF/LOW
  duration = pulseIn(echo, HIGH);  //reads pulse of echo is ON/HIGH
  distance = ((duration*0.0343)/2) ;  //formula

  LCD.setCursor(0,1);   //Set Cursor to first column of second row
  LCD.print(distance); //Print measured distance
  LCD.print(" cm");  //Print units.
  delay(250);
}
```

Procedure:

1) Connect the circuit as per the diagram.
2) Type the Arduino Sketch 1 in the IDE.
3) Upload the program to Arduino.
4) Place an object in front of the sensor and observe the measured distance on the LCD.
5) Measure the actual distance of the of the object and sensor using a meter scale and record the reading.
6) Change the position of the object and repeat steps 4 and 5.

Observation

Obs. No	Actual Distance (A) (cm)	Measured Distance (M) (cm)	% error $= \frac{\lvert A - M \rvert}{A}$
1.			
2.			
3.			
4.			

Result

The distance of an object from the ultrasonic sensor was successfully measured and displayed on the LCD. The observed error percentage was found to be within acceptable limits.

Practice Task

Modify the code to display the distance in both centimetres and inches on the LCD.

---

6. LDR with Arduino

Aim: Use an LDR (photoresistor) with a voltage divider to detect ambient light and switch an LED using Arduino.

Components: Arduino Uno (or compatible)**,** Breadboard, Jumper wires, 150Ω & 10kΩ resistor, LDR and computer with Arduino IDE.

Theory: A Light Dependent Resistor (LDR) is a photo-sensitive component whose resistance changes with incident light.

- **Bright light → Low resistance**
- **Darkness → High resistance**

When connected as a voltage divider with a fixed resistor (10 kΩ), the voltage at the junction varies with light intensity.

This analog voltage is fed to Arduino's A0 pin, which converts it to a digital value (0 – 1023). By comparing this value to a preset threshold (e.g., 300), the Arduino decides whether to turn an LED ON (dark) or OFF (bright).

Voltage divider formula:

$$V_{out} = V_{cc} \times \frac{R_L}{R_L + R_k}$$

Where $R_L \rightarrow LDR$, $R_k = 10\ k\Omega$

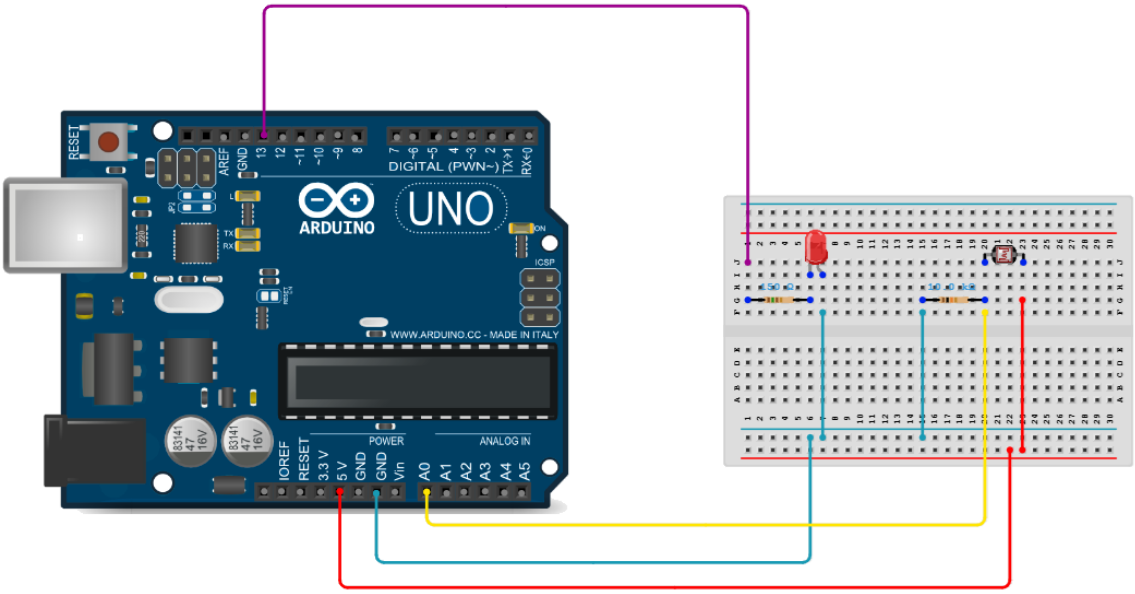


Figure 3: Connection of LDR and LED

Arduino Sketch

```
const int ledPin = 13;
const int ldrPin = A0;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  pinMode(ldrPin, INPUT);
}

void loop() {
  int ldrStatus = analogRead(ldrPin);  // Read analog value (0-1023)

  Serial.print("LDR Value: ");
  Serial.println(ldrStatus);

  if (ldrStatus <= 300) {           // Threshold value for darkness
    digitalWrite(ledPin, HIGH);        // LED ON
    Serial.println("LDR is DARK, LED is ON");
  }
  else {
    digitalWrite(ledPin, LOW);         // LED OFF
```

```
    Serial.println("---------------");
  }

  delay(500);
}
```

Procedure:

1) Assemble the circuit as shown in the circuit diagram.
2) Connect the Arduino UNO to the computer via USB.
3) Open Arduino IDE, upload the Arduino sketch to the board.
4) Open the Serial Monitor at 9600 baud rate.
5) Note the LDR values under different light conditions:
   - Bright (torch or sunlight)
   - Covered/dark
   - Dim light
6) Observe LED behaviour and record both LDR reading and LED state.

Observation

Obs. No	Light Condition	LDR reading	LED Status	Serial Output
1.	Bright		ON/OFF	
2.	Covered/dark		ON/OFF	
3.	Dim light		ON/OFF	

Result

The LED glows when the LDR senses darkness and turns OFF in bright light. Thus, the LDR successfully detects variations in light intensity using Arduino.

Practice Task

1) Perform a simple experiment to quantify how the LDR reading changes with light distance.
2) Design a night-lamp system for homes using Arduino UNO, an LDR, and an LED strip.

---

7. PIR Motion Sensor with Arduino

Aim: To detect motion using a **PIR (Passive Infrared) sensor** and control an LED using Arduino.

Components: Arduino Uno (or compatible)**,** Breadboard, Jumper wires, 220Ω resistor, PIR Motion Sensor (HC-SR501 or equivalent) and computer with Arduino IDE.

Theory: A PIR (Passive Infrared) sensor detects motion by sensing changes in infrared radiation emitted by objects (such as the human body).

- When no motion is detected → Output = LOW (0 V)
- When motion is detected → Output = HIGH (~5 V)

The PIR sensor has three pins:

- VCC → +5V
- OUT → Digital signal to Arduino
- GND → Ground

When motion is detected, the PIR sensor sends a HIGH signal to Arduino, which can be used to trigger an LED or alarm.

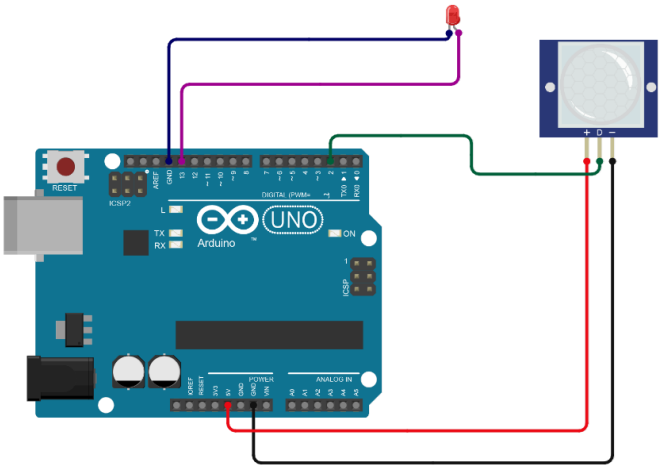


Figure 4: Connection of PIR and LED

Arduino Sketch

```
int pirPin = 2;      // PIR output pin
int ledPin = 13;     // LED pin
int motionState = 0;

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  motionState = digitalRead(pirPin);

  if (motionState == HIGH) {
```

```
    digitalWrite(ledPin, HIGH);   // LED ON (Alarm)
    Serial.println("Motion Detected! Alarm ON");
  } else {
    digitalWrite(ledPin, LOW);    // LED OFF
    Serial.println("No Motion");
  }

  delay(200);
}
```

Procedure:

1) Connect the PIR sensor and LED as shown in the circuit diagram.
2) Upload the program to Arduino using Arduino IDE.
3) Open the Serial Monitor (9600 baud).
4) Wait 30–60 seconds for the PIR sensor to stabilize.
5) Move your hand/body in front of the sensor:
6) LED should turn ON (alarm condition)
7) Serial Monitor displays *"Motion Detected! Alarm ON"*
8) Stop movement and observe LED turning OFF.
9) Adjust the sensitivity and delay knobs on the PIR module and repeat observations.
10)

Observation

Obs. No	Condition	PIR Output	LED Status	Serial Output
1.	No Motion Present	High/Low	ON/OFF	
2.	Motion Detected			
3.	Continuous movement			
4.	After Delay (no motion)			

Result

The PIR sensor successfully detects motion and triggers the LED alarm system using Arduino. The system responds reliably to changes in infrared radiation.

Practice Task

1) When motion is detected, the LED should remain ON for 5 seconds, even if motion stops.

2) Add a buzzer along with the LED for audible alert.

3) Design and implement a smart lighting system using Arduino by interfacing an LDR for automatic night detection and integrating a PIR sensor to activate lighting only when motion is detected in low-light conditions.

---

8. Arduino Project: PCB Design, Assembly and Testing

Aim: To design, assemble, and test an Arduino-based project by creating a PCB using EasyEDA and implementing the corresponding Arduino program.

Components: Arduino Uno (or compatible), Computer/Laptop with internet access, EasyEDA, Arduino IDE, Prefabricated PCB (based on designed project), Sensors (LDR/PIR/others as per project), Electronic components (resistors, LEDs, connectors, etc.), Soldering iron, solder wire, flux, Breadboard and jumper wires (for testing, if required)

Theory: The development of an Arduino-based system involves three major stages:

1. **Design** – Creating a circuit and PCB layout
2. **Implementation** – Assembling and soldering components
3. **Testing** – Uploading code and verifying system performance

PCB design transforms a prototype circuit into a compact, durable, and efficient system, while Arduino programming enables interaction between hardware components and control logic.

Procedure

1) **Project Selection & Circuit Design:**
Explore suitable Arduino-based projects by referring to course materials or surfing the internet. Select a project aligned with the skills learned. Draw the circuit diagram showing all connections. Draw the circuit diagram showing all connections between Arduino, sensors, and output components.

2) **PCB Design using EasyEDA**
Create the schematic in EasyEDA, assign footprints, convert it into a PCB layout, route connections, and perform DRC to ensure correctness.

3) **PCB Assembly (Hands-On)**
Assemble and solder the prefabricated PCB with Arduino headers, sensors, and other components.

4) **Code Upload & Testing**
Upload and modify the Arduino program using Arduino IDE. Test the system by verifying inputs and outputs.

5) **Demonstration**
   Debug if required and demonstrate the working of the PCB-based Arduino project.

Record Your Work (Fill-in Section)

1. Project Selection & Circuit Design

- **Project Title:** _____
- **Objective of the Project:** _____

  _____
- **Components Used:** _____

  _____
- **Circuit Diagram (Draw neatly in the space below):**

2. PCB Design using EasyEDA

- **Software Used: EasyEDA**
- **Number of Components in Design:** _____
- **Number of Connections (Nets):** _____
- **PCB Layers Used (Top/Bottom):** _____
- **DRC Errors Found (if any):** _____
- **Corrections Made:** _____

  _____

3. PCB Assembly (Hands-On)

- **PCB Assembly Completed (Yes/No):** _____
- **Components Soldered:** _____

  _____
- **Issues Faced During Soldering:** _____

_____

_____

4. Code Upload & Testing

- **Code Uploaded Successfully (Yes/No):** _____
- **Type of Program Used:** _____
- **Inputs Given (Sensor Conditions):** _____

  _____

  _____

- **Outputs Observed:** _____

  _____

  _____

5. Debugging & Demonstration

- **Errors Encountered:** _____

  _____

  _____

- **Corrections Made:** _____

  _____

  _____

- **Final Working Status (Working/Not Working):** _____

Result

The Arduino-based project was successfully designed, assembled on a PCB, and tested, demonstrating proper integration of hardware and software.

Student Reflection:

- What did you learn from this experiment?

  _____

  _____

  _____

  _____

- What improvements can be made to your project?